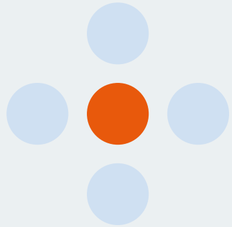
15 July 2026

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Stencil Computation

Parallelization and Scaling



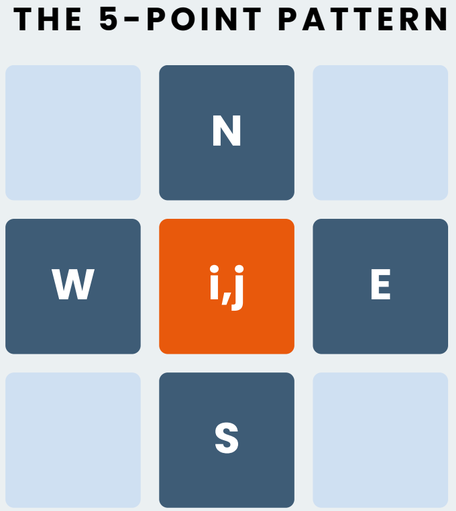
Presented By Giacomo Serafini

01 · THE PROBLEM

A Two-Dimensional Five-Point Stencil

At every grid point, the new value depends on the old value of the point itself and its four direct neighbours:

$$new_{i,j} = \alpha \cdot old_{i,j} + \frac{1-\alpha}{4}\left(old_{i-1,j} + old_{i+1,j} + old_{i,j-1} + old_{i,j+1}\right)$$



- **Independent within a timestep**

All interior points at iteration t depend only on iteration t-1 → they can be updated concurrently

- **Two levels of parallelism**

Shared memory within a node (OpenMP, implicit communication) and distributed memory across nodes (MPI, explicit halo exchange)

Bandwidth Sets the Limit

Considering one grid-point update in isolation:

- **5 loads + 1 store**

48 bytes of memory traffic per point (center + 4 neighbours, one write)

- **6 FLOPs**

1 multiplication + 3 additions + 1 multiplication + 1 addition

AI = 6 FLOPs / 48 B = 0.125 FLOPs/B

Arithmetic intensity this low means throughput is capped by how fast data can be delivered from memory, not by peak FLOP/s

42 GFLOP/s
Theoretical peak double-precision throughput, per EPYC 7H12 core

3.2 GB/s
Memory bandwidth available per core once shared across all 128 cores of a node

0.40 GFLOP/s
Compute throughput actually sustainable once fed at that bandwidth (3.2 × 0.125)

≈100x gap between peak compute and what memory can feed
→ optimise data movement first

03 · SERIAL OPTIMISATION

Maximising Locality Before Parallelising

Cache Blocking (Tiling)

Row-major sweeps have poor temporal locality: neighbouring rows fall out of L1 before being reused
→ partition the domain into square tiles to keep each tile's working set resident

Pointer Aliasing

old/new buffers marked *restrict*, telling the compiler the regions never overlap
→ more aggressive optimisations

Memory Alignment

Both planes allocated with *posix_memalign(..., 64, ...)*: 64-byte aligned buffers to avoid misalignment penalties

Compiler-Level Opt.

- Stencil coefficients hoisted out of hot loops as precomputed constants
- Divisions replaced with equivalent multiplications

OpenMP: Threads, Tiles, and NUMA Locality

Interior / Border Split

update_plane_inside() carries the bulk
update_plane_border() handles the thin boundary
→ both parallelised

First-Touch Initialisation

Grid is zeroed with the same tiled distribution used at compute time
→ pages land in the NUMA domain of the thred what will use them

collapse(2), schedule(static)

- The two tile-index loops merge into one iteration space
- Static scheduling minimise overhead since every tile costs the same

OMP_PLACES=cores OMP_PROC_BIND=close

- Threads pinned to physical cores
- Neighbouring threads kept on nearby cores

→ preserve first-touch locality

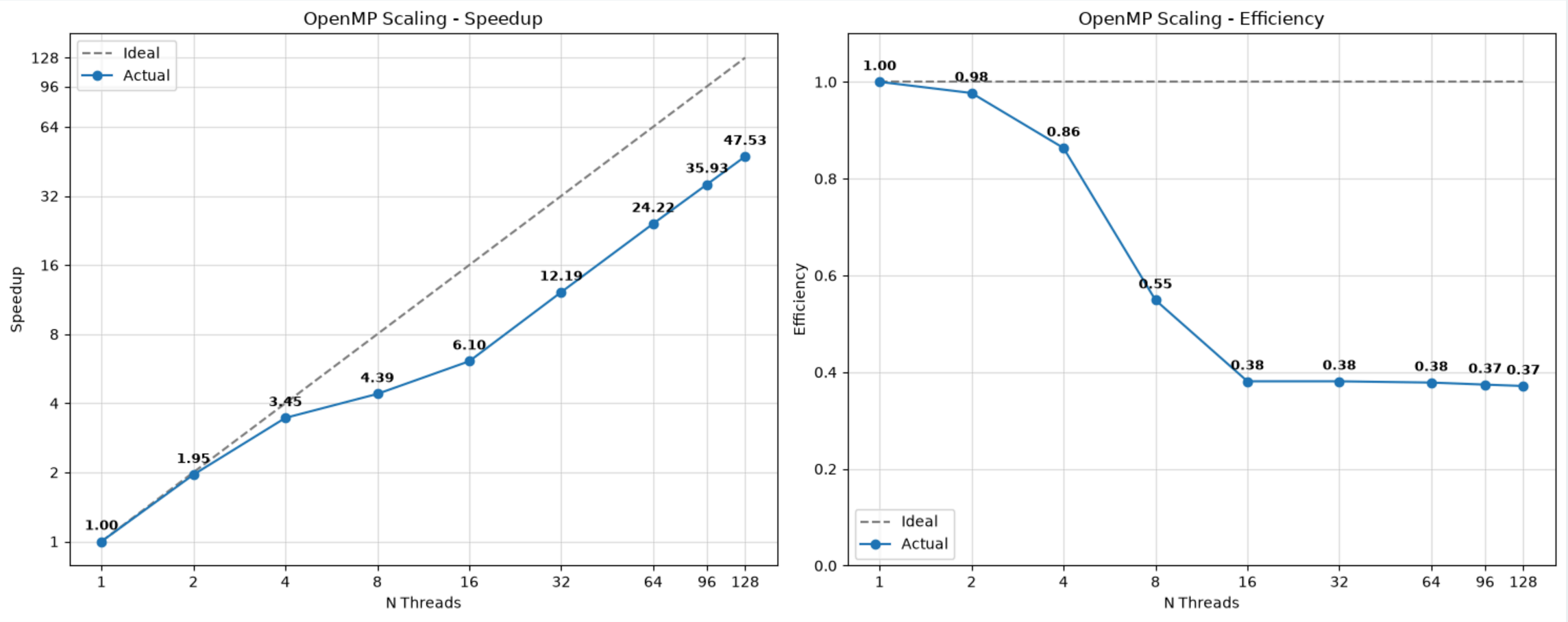
#pragma omp simd

Inner loop vectorised explicitly
→ with *restrict* and aligned buffers, the compiler emit efficient SIMD instructions

reduction(+) for energy

Global energy summation gives each thread a private partial sum, combined at the end with no synchronisation overhead

Thread Scaling on a Single EPYC Node



1-128 OpenMP threads · single MPI rank · one Orfeo EPYC node (8 NUMA domains × 16 cores)

Reading the Curve: NUMA Bandwidth Saturation

> 85%
efficiency up to 4 threads:
bandwidth and compute still unsaturated

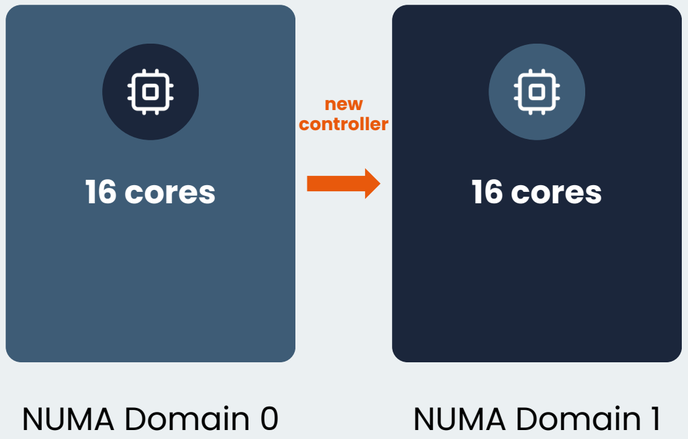
~ 38%
efficiency beyond 16 threads

47.5×
speedup at 128 threads on a full node

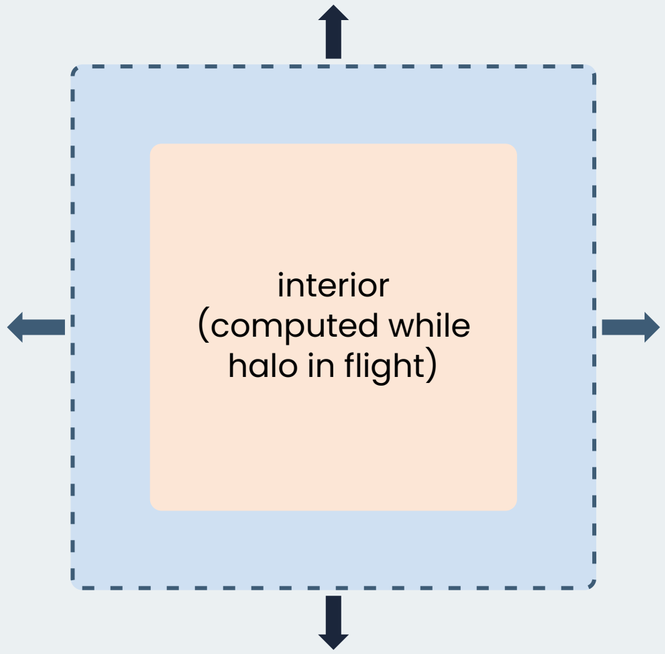
Within a single NUMA domain, all threads share one memory controller → the bandwidth-bound kernel saturates it quickly, so efficiency collapses before 16 threads are reached

→ crossing a new domain adds an independent memory controller: the achievable bandwidth genuinely increases and speedup resumes

CROSSING A DOMAIN BOUNDARY



Halo Exchange with Compute/Communication Overlap



N / S rows — zero-copy pointer alias (already contiguous)

E / W columns — packed into contiguous buffers before send

- **Non-Blocking *Isend* / *Irecv***

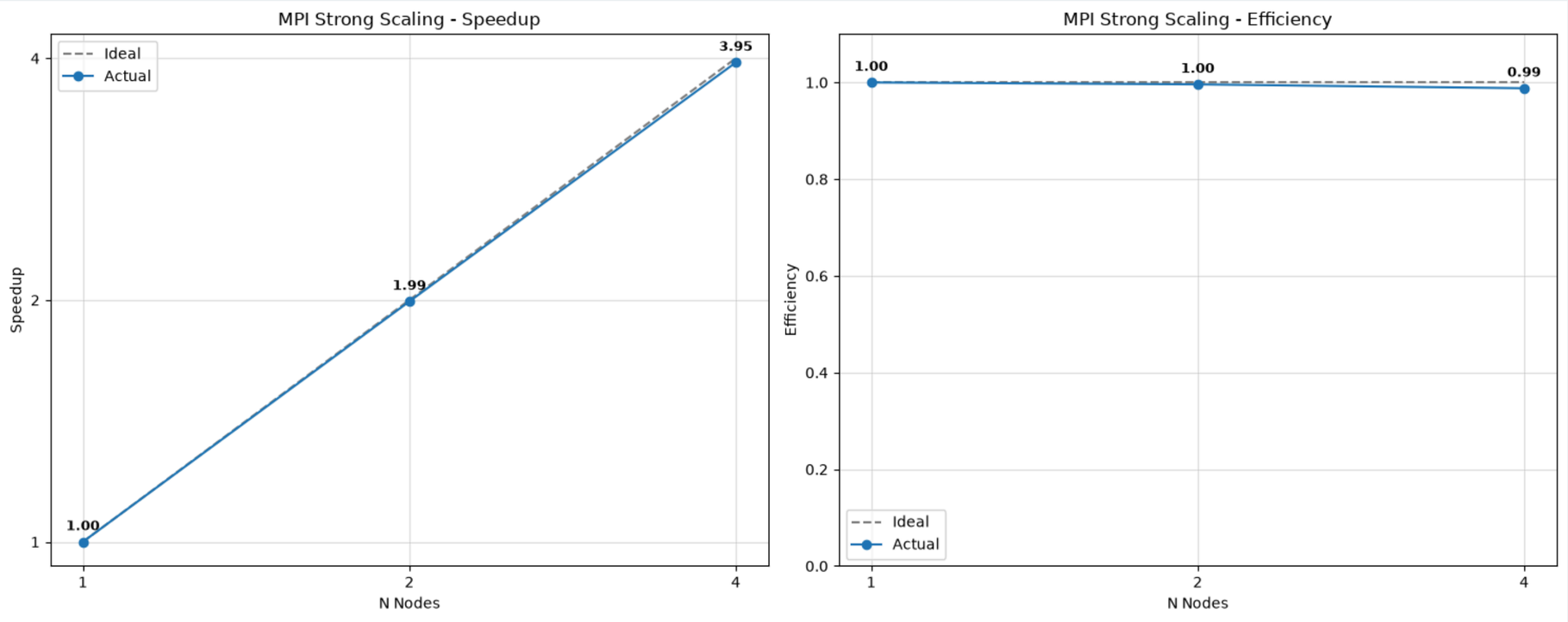
All four requests posted before the stencil update begins
→ interior points compute while transfers
→ *MPI_Waitall* before computing borders

- ***MPI_Wtime*, max & average**

Compute and communication timed separately
→ *MPI_MAX* exposes critical path, *MPI_SUM/avg* shows load balance

06 · SCALING RESULTS

MPI Strong Scaling



0.99
efficiency at 32 ranks / 4 nodes

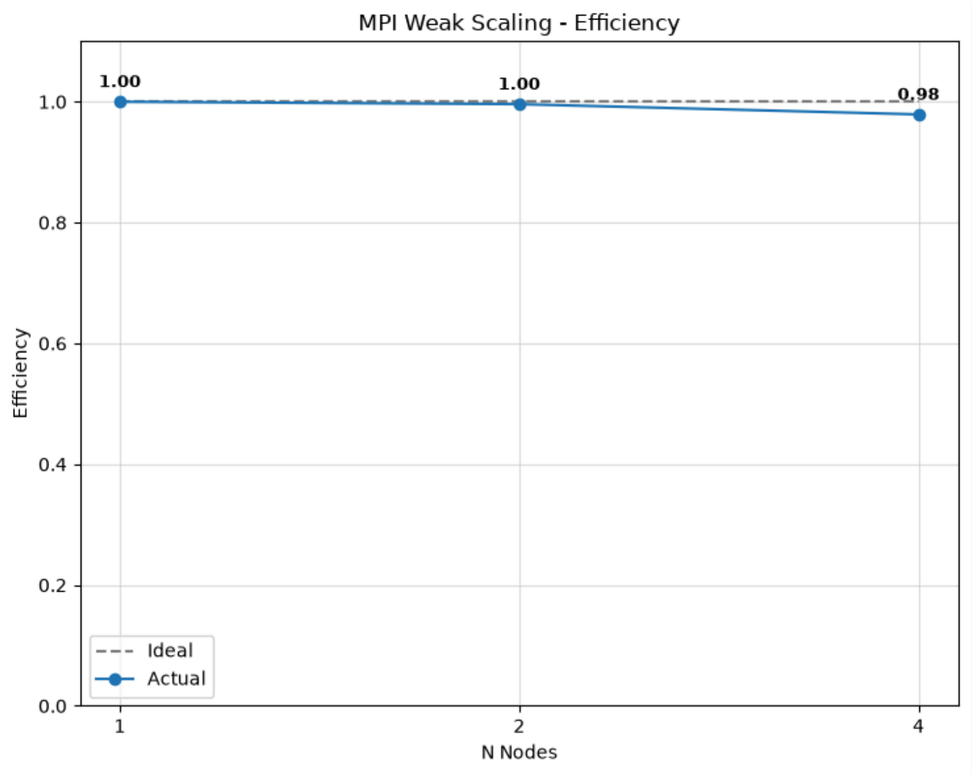
One rank per NUMA domain means adding ranks adds independent compute-and-bandwidth units

64000² grid · 8, 16, 32 MPI ranks (1, 2, 4 nodes) × 16 OpenMP threads/rank · avg over 3 reps

Overlapping halo exchange with interior compute keeps communication cost negligible even as subdomains shrink

06 · SCALING RESULTS

MPI Weak Scaling



Nodes	Tasks	Grid side	Time (s)	Efficiency
1	8	40000	13.994	1.0000
2	16	56576	14.047	0.996
4	32	80000	14.290	0.979

With subdomains fixed, ideal efficiency confirms no additional bottleneck emerges purely from involving more nodes

starting 40000² grid · 8, 16, 32 MPI ranks (1, 2, 4 nodes) × 16 OpenMP threads/rank · avg over 3 reps

→ constant work per rank

15 July 2026

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Thank You

Presented By Giacomo Serafini